Graph Algorithms

This manual covers the graph algorithm capabilities of the Graph OLAP SDK, including native Ryugraph algorithms and NetworkX integration for advanced analytics.

1. Algorithm Overview

The Graph OLAP Platform provides different algorithm systems depending on the database backend:

Ryugraph Instances:

1. **Native Ryugraph Algorithms** (`conn.algo`) - High-performance algorithms that run directly in the database engine using the KuzuDB algo extension
2. **NetworkX Algorithms** (`conn.networkx`) - Access to 100+ algorithms from the NetworkX library via dynamic introspection

FalkorDB Instances:

1. **Native FalkorDB Algorithms** (`conn.algo`) - Algorithms via Cypher procedures (`CALL algo.xxx()`). Available algorithms: `pagerank` , `betweenness` , `wcc` , `cdlp`

> **Important:** FalkorDB does **not** support NetworkX integration. Only native Cypher procedures are available. Ryugraph supports both native algorithms and the full NetworkX library (100+ algorithms).

Execution Model

All algorithms follow an asynchronous execution model:

1. **Lock Acquisition** - The instance acquires an exclusive lock
2. **Algorithm Execution** - The algorithm runs against the graph data
3. **Result Writeback** - Results are written to node/edge properties
4. **Lock Release** - The lock is released for other operations

```
# Synchronous (default) — blocks until completion
exec = conn.algo.pagerank("Customer", "pr_score")

# Asynchronous — returns immediately, poll for status
exec = conn.algo.pagerank("Customer", "pr_score", wait=False)
while exec.status == "running":
    time.sleep(2)
    exec = conn.algo.status(exec.execution_id)
```

Result Storage

Algorithm results are stored as node properties and can be queried using Cypher:

```
# Run PageRank and store in 'importance' property
conn.algo.pagerank("Customer", "importance")

# Query results using Cypher
result = conn.query("""
    MATCH (c:Customer)
    WHERE c.importance > 0.01
    RETURN c.name, c.importance
    ORDER BY c.importance DESC
    LIMIT 10
""")
```

Lock Mechanism

Only one algorithm can run at a time per instance:

```
try:
    conn.algo.louvain("Customer", "community")
except ResourceLockedError as e:
    print(f"Instance locked by {e.holder_username} running {e.algorithm_name}")
```

---

2. Native Ryugraph Algorithms (conn.algo)

Native algorithms run directly in the Ryugraph/KuzuDB engine using the algo extension, optimized for performance.

Algorithm Discovery

```
# List all available native algorithms
algos = conn.algo.algorithms()
for algo in algos:
    print(f"{algo['name']}: {algo['description']}")

# Filter by category
centrality_algos = conn.algo.algorithms(category="centrality")
community_algos = conn.algo.algorithms(category="community")

# Get detailed information about an algorithm
info = conn.algo.algorithm_info("pagerank")
print(f"Parameters: {info['parameters']}")
```

Generic Execution

```
exec = conn.algo.run(
    "pagerank",
    node_label="Customer",
    property_name="pr_score",
    edge_type="KNOWS",
    params={"damping_factor": 0.85, "max_iterations": 100},
    timeout=300,
    wait=True
)
print(f"Nodes updated: {exec.nodes_updated}, Duration: {exec.duration_ms}ms")
```

PageRank

Computes node importance based on link structure:

```
exec = conn.algo.pagerank(
    node_label="Customer",
    property_name="pr_score",
    edge_type="TRANSACTS_WITH",
    damping=0.85,
    max_iterations=100,
    tolerance=1e-6
)
```

Use Cases: Identifying influencers, ranking entities, fraud detection

Weakly Connected Components (WCC)

Finds groups of connected nodes (treating edges as undirected):

```
exec = conn.algo.connected_components(
    node_label="Customer",
    property_name="component_id",
    edge_type="KNOWS"
)
# Alternative shorthand
exec = conn.algo.wcc("Customer", "component_id")
```

Use Cases: Finding isolated segments, network segmentation

Strongly Connected Components (SCC)

Finds groups where every pair is mutually reachable:

```
exec = conn.algo.scc(
    node_label="Account",
    property_name="scc_id",
    edge_type="TRANSFERS_TO"
)

# Kosaraju variant (better for sparse graphs)
exec = conn.algo.scc_kosaraju("Account", "scc_id", edge_type="TRANSFERS_TO")
```

Use Cases: Detecting circular patterns, finding tightly coupled entities

Louvain Community Detection

Hierarchical clustering that maximizes modularity:

```
exec = conn.algo.louvain(
    node_label="Customer",
    property_name="community_id",
    edge_type="KNOWS",
    resolution=1.0  # Higher = more communities
)
```

Use Cases: Customer segmentation, fraud ring detection

K-Core Decomposition

Finds nodes connected to at least k other nodes:

```
exec = conn.algo.kcore(
    node_label="Customer",
    property_name="k_degree",
    edge_type="KNOWS"
)
```

Use Cases: Cohesive group detection, network resilience analysis

Label Propagation

Fast community detection via neighbor label adoption:

```
exec = conn.algo.label_propagation(
    node_label="Customer",
    property_name="label",
    edge_type="KNOWS",
```

```
    max_iterations=100
)
```

Triangle Count

Counts triangles each node participates in:

```
exec = conn.algo.triangle_count(
    node_label="Customer",
    property_name="triangles",
    edge_type="KNOWS"
)
```

Use Cases: Measuring clustering, identifying tight neighborhoods

Shortest Path

Find the shortest path between two nodes:

```
exec = conn.algo.shortest_path(
    source_id="customer_001",
    target_id="customer_050",
    relationship_types=["KNOWS", "WORKS_WITH"],
    max_depth=6
)

if exec.result and exec.result.get("found"):
    print(f"Path: {exec.result['path']}, Length: {exec.result['length']}")
```

---

3. NetworkX Algorithms (conn.networkx)

> **Note:** NetworkX algorithms are only available for Ryugraph instances. FalkorDB instances use native Cypher procedures instead. See Section 4: FalkorDB Algorithms for FalkorDB-specific documentation.

Access to 100+ algorithms from NetworkX through dynamic introspection.

How NetworkX Integration Works

1. Graph data is extracted from Ryugraph to NetworkX format
2. The algorithm runs in NetworkX (Python)
3. Results are written back to Ryugraph node properties

Algorithm Discovery

```python
# List all available NetworkX algorithms
algos = conn.networkx.algorithms()
print(f"Found {len(algos)} algorithms")

# Filter by category
centrality = conn.networkx.algorithms(category="centrality")
community = conn.networkx.algorithms(category="community")
clustering = conn.networkx.algorithms(category="clustering")

# Get detailed algorithm information
info = conn.networkx.algorithm_info("betweenness_centrality")
```

Generic Execution

```python
exec = conn.networkx.run(
    "katz_centrality",
    node_label="Customer",
    property_name="katz_score",
    params={"alpha": 0.1, "beta": 1.0},
    timeout=300
)
```

Centrality Algorithms

```python
# Degree Centrality - connection count
exec = conn.networkx.degree_centrality("Customer", "degree_cent")

# Betweenness Centrality - network brokers
exec = conn.networkx.betweenness_centrality("Customer", "betweenness")
exec = conn.networkx.betweenness_centrality("Customer", "betw_approx", k=100)  # Approximate

# Closeness Centrality - network position
exec = conn.networkx.closeness_centrality("Customer", "closeness")

# Eigenvector Centrality - influence
exec = conn.networkx.eigenvector_centrality("Customer", "eigenvector", max_iter=100)
```

Clustering Coefficient

```python
exec = conn.networkx.clustering_coefficient("Customer", "clustering")
```

Advanced Algorithms via run()

```
# Katz Centrality
exec = conn.networkx.run("katz_centrality", "Customer", "katz", params={"alpha": 0.1})

# Harmonic Centrality
exec = conn.networkx.run("harmonic_centrality", "Customer", "harmonic")

# Load Centrality
exec = conn.networkx.run("load_centrality", "Customer", "load")
```

---

4. FalkorDB Algorithms (conn.algo)

FalkorDB provides native graph algorithms via Cypher procedures. Unlike Ryugraph, FalkorDB does **not** support NetworkX integration. All algorithms execute asynchronously with status polling, using the same `conn.algo` interface.

Available Algorithms

Algorithm	Method	Cypher Procedure	Category	Description
PageRank	`pagerank()`	`pagerank.stream`	Centrality	Node importance based on incoming links
Betweenness	`betweenness()`	`algo.betweenness`	Centrality	Bridge node identification
WCC	`wcc()`	`algo.WCC`	Community	Weakly connected component IDs
CDLP	`cdlp()`	`algo.labelPropagation`	Community	Community detection via label propagation

Algorithm Discovery

```
# List all available FalkorDB algorithms
algos = conn.algo.algorithms()
for algo in algos:
```

```
    print(f"{algo['name']}: {algo['description']}")

# Get detailed information about an algorithm
info = conn.algo.algorithm_info("pagerank")
print(f"Cypher procedure: {info['cypher_procedure']}")
print(f"Parameters: {info['parameters']}")
```

PageRank

Computes node importance based on link structure:

```
exec = conn.algo.pagerank(
    result_property="pr_score",
    node_labels=["Customer"],
    relationship_types=["TRANSACTS_WITH"],
    timeout_ms=300000
)
print(f"Nodes updated: {exec.nodes_updated}")
```

Betweenness Centrality

Identifies bridge nodes that control information flow:

```
exec = conn.algo.betweenness(
    result_property="betweenness_score",
    node_labels=["Customer"],
    relationship_types=["KNOWS"],
    timeout_ms=3600000  # 1 hour - O(V*E) complexity
)
```

Weakly Connected Components (WCC)

Finds groups of connected nodes (treating edges as undirected):

```
exec = conn.algo.wcc(
    result_property="component_id",
    node_labels=["Customer"],
    relationship_types=["KNOWS"]
)
```

Community Detection Label Propagation (CDLP)

Fast community detection via neighbor label adoption:

```
exec = conn.algo.cdlp(
    result_property="community_id",
    node_labels=["Customer"],
    relationship_types=["KNOWS"],
    max_iterations=10
)
```

Pathfinding (Synchronous)

FalkorDB pathfinding algorithms run synchronously via Cypher queries (no async execution needed):

```
# Breadth-First Search
result = conn.query("""
    MATCH path = algo.BFS((a:Person {id: 'A'}), (b:Person {id: 'B'}))
    RETURN path
""")

# Shortest Path
result = conn.query("""
    MATCH path = algo.shortestPath((a:Person {id: 'A'}), (b:Person {id: 'B'}))
    RETURN path
""")
```

Result Storage

Algorithm results are written to node properties and can be queried using Cypher:

```
# Run PageRank
conn.algo.pagerank(result_property="importance", node_labels=["Customer"])

# Query results
result = conn.query("""
    MATCH (c:Customer)
    WHERE c.importance > 0.01
    RETURN c.name, c.importance
    ORDER BY c.importance DESC
    LIMIT 10
""")
```

---

5. Algorithm Results

AlgorithmExecution Object

```python
exec = conn.algo.pagerank("Customer", "pr_score")

# Execution metadata
print(f"Execution ID: {exec.execution_id}")
print(f"Algorithm: {exec.algorithm}")
print(f"Type: {exec.algorithm_type}")  # "native" or "networkx"

# Status tracking
print(f"Status: {exec.status}")  # pending, running, completed, failed, cancelled
print(f"Started: {exec.started_at}")
print(f"Completed: {exec.completed_at}")

# Results
print(f"Nodes Updated: {exec.nodes_updated}")
print(f"Duration: {exec.duration_ms}ms")

# Error information
if exec.status == "failed":
    print(f"Error: {exec.error_message}")
```

Status Values

Status	Description
`pending`	Queued, not yet started
`running`	Currently executing
`completed`	Successfully finished
`failed`	Execution failed
`cancelled`	Cancelled by user

Polling for Completion

```python
exec = conn.algo.louvain("Customer", "community", wait=False)

while exec.status in ("pending", "running"):
    time.sleep(2)
    response = conn._client.get(f"/algo/status/{exec.execution_id}")
    exec = AlgorithmExecution.from_api_response(response.json())

print(f"Completed: {exec.nodes_updated} nodes in {exec.duration_ms}ms")
```

Querying Results via Cypher

```python
# After running community detection
conn.algo.louvain("Customer", "community_id")
```

```python
# Query community membership
result = conn.query("""
    MATCH (c:Customer)
    RETURN c.community_id, count(*) AS members
    ORDER BY members DESC
""")

# Combine multiple algorithm results
conn.algo.pagerank("Customer", "importance")

result = conn.query("""
    MATCH (c:Customer)
    RETURN c.community_id, avg(c.importance) AS avg_importance
    ORDER BY avg_importance DESC
""")
```

---

6. Algorithm Quick Reference

Ryugraph Native Algorithms (conn.algo)

Algorithm	Category	Method	Use Case
PageRank	Centrality	`pagerank()`	Node importance
WCC	Community	`connected_components()`	Connected groups
SCC	Community	`scc()`	Strongly connected groups
SCC Kosaraju	Community	`scc_kosaraju()`	SCC for sparse graphs
Louvain	Community	`louvain()`	Community detection
K-Core	Community	`kcore()`	Cohesive groups
Label Propagation	Community	`label_propagation()`	Fast communities
Triangle Count	Community	`triangle_count()`	Clustering measurement
Shortest Path	Pathfinding	`shortest_path()`	Path finding

FalkorDB Native Algorithms (conn.algo)

Algorithm	Category	Method	Cypher Procedure	Use Case
PageRank	Centrality	`pagerank()`	`pagerank.stream`	Node importance

Algorithm	Category	Method	Cypher Procedure	Use Case
Betweenness	Centrality	`betweenness()`	`algo.betweenness`	Network brokers
WCC	Community	`wcc()`	`algo.WCC`	Connected groups
CDLP	Community	`cdlp()`	`algo.labelPropagation`	Fast communities
BFS	Pathfinding	Cypher query	`algo.BFS`	Path finding
Shortest Path	Pathfinding	Cypher query	`algo.shortestPath`	Path finding

NetworkX Algorithms (conn.networkx) - Ryugraph Only

Algorithm	Category	Method	Use Case
Degree Centrality	Centrality	`degree_centrality()`	Connection count
Betweenness Centrality	Centrality	`betweenness_centrality()`	Network brokers
Closeness Centrality	Centrality	`closeness_centrality()`	Network position
Eigenvector Centrality	Centrality	`eigenvector_centrality()`	Influence
Clustering Coefficient	Clustering	`clustering_coefficient()`	Local clustering
Katz Centrality	Centrality	`run("katz_centrality")`	Influence with decay

Parameter Reference

Ryugraph PageRank: `damping` (0.85), `max_iterations` (100), `tolerance` (1e-6)

Louvain: `resolution` (1.0), `max_phases` (20), `max_iterations` (20)

WCC/SCC: `max_iterations` (100)

FalkorDB CDLP: `max_iterations` (10)

---

7. Practical Examples

Customer Influence Analysis

```
# Calculate multiple centrality measures
conn.algo.pagerank("Customer", "pagerank", edge_type="TRANSACTS_WITH")
conn.networkx.betweenness_centrality("Customer", "betweenness")
conn.networkx.eigenvector_centrality("Customer", "eigenvector")

# Find influential customers
result = conn.query("""
    MATCH (c:Customer)
    WHERE c.pagerank > 0.01 AND c.betweenness > 0.05
    RETURN c.name, c.pagerank, c.betweenness, c.eigenvector
    ORDER BY c.pagerank DESC
    LIMIT 20
""")
```

Fraud Ring Detection

```
conn.algo.louvain("Account", "community_id", edge_type="TRANSFERS_TO")
conn.algo.scc("Account", "scc_id", edge_type="TRANSFERS_TO")

# Find suspicious circular patterns
result = conn.query("""
    MATCH (a:Account)
    WITH a.community_id AS community, a.scc_id AS scc, count(*) AS ring_size
    WHERE ring_size >= 3
    RETURN community, scc, ring_size
    ORDER BY ring_size DESC
""")
```

Network Segmentation

```
conn.algo.connected_components("Customer", "segment_id")

result = conn.query("""
    MATCH (c:Customer)
    RETURN c.segment_id, count(*) AS customers, sum(c.total_balance) AS total_balance
    ORDER BY total_balance DESC
""")
```

Combined Analysis Pipeline

```
algorithms = [
    ("pagerank", "pr_score", {"damping": 0.85}),
    ("louvain", "community_id", {"resolution": 1.0}),
    ("wcc", "component_id", {}),
]

for algo_name, prop_name, params in algorithms:
    exec = conn.algo.run(algo_name, "Customer", prop_name, edge_type="KNOWS", params=params)
    print(f"{algo_name}: {exec.nodes_updated} nodes in {exec.duration_ms}ms")
```

```
# Summary
summary = conn.query("""
    MATCH (c:Customer)
    RETURN count(*) AS total, count(DISTINCT c.community_id) AS communities
""")
```

---

8. Performance Considerations

Native vs NetworkX

Aspect	Native	NetworkX
Performance	Fast (in-DB)	Slower (data transfer)
Algorithms	8 core	100+ algorithms
Large Graphs	Recommended	Use subgraph filtering
Memory	Low overhead	Requires graph in memory

Best Practices

1. **Use native algorithms when available** - Much faster for large graphs
2. **Set appropriate timeouts** - Prevent resource exhaustion
3. **Filter with subgraphs** - For NetworkX on large graphs
4. **Clean up properties** - Remove unused result properties

```
# Remove old algorithm properties
conn.query("MATCH (c:Customer) REMOVE c.old_pagerank, c.old_community")
```

---

9. Troubleshooting

Common Issues

ResourceLockedError:

```
while True:
    try:
        conn.algo.pagerank("Customer", "pr")
```

```
        break
    except ResourceLockedError:
        time.sleep(5)
```

AlgorithmTimeoutError:

```
exec = conn.algo.pagerank("Customer", "pr", timeout=600)  # 10 minutes
```

AlgorithmNotFoundError:

```
native = conn.algo.algorithms()
networkx = conn.networkx.algorithms()
print([a['name'] for a in native])
```

Debugging

```
# Verify results were written
result = conn.query("""
    MATCH (c:Customer)
    WHERE c.pr_score IS NOT NULL
    RETURN count(*) AS nodes_with_results
""")
print(f"Nodes with results: {result.scalar()}")

# Check for NULL values (algorithm may not cover all nodes)
result = conn.query("""
    MATCH (c:Customer)
    WHERE c.pr_score IS NULL
    RETURN count(*) AS nodes_without_results
""")
print(f"Nodes without results: {result.scalar()}")

# Inspect schema after algorithm execution
schema = conn.schema()
print(f"Customer properties: {schema.node_labels.get('Customer', [])}")
```

Error Handling Best Practices

```
from graph_olap.exceptions import (
    AlgorithmFailedError,
    AlgorithmTimeoutError,
    AlgorithmNotFoundError,
    ResourceLockedError
)

def run_algorithm_safely(conn, algo_func, *args, max_retries=3, **kwargs):
    """Run algorithm with retry logic and proper error handling."""
    for attempt in range(max_retries):
```

```
        try:
            return algo_func(*args, **kwargs)
        except ResourceLockedError as e:
            print(f"Attempt {attempt + 1}: Instance locked by {e.holder_username}")
            if attempt < max_retries - 1:
                time.sleep(10)
            else:
                raise
        except AlgorithmTimeoutError:
            print(f"Attempt {attempt + 1}: Timeout, retrying with longer timeout")
            kwargs['timeout'] = kwargs.get('timeout', 300) * 2
        except AlgorithmFailedError as e:
            print(f"Algorithm failed: {e}")
            raise

# Usage
exec = run_algorithm_safely(
    conn, conn.algo.pagerank,
    "Customer", "pr_score",
    edge_type="KNOWS"
)
```

---

10. Algorithm Categories Explained

Centrality Algorithms

Centrality algorithms measure the importance or influence of nodes in a graph. Different centrality measures capture different aspects of importance:

Measure	What it Captures	When to Use
PageRank	Overall importance via link structure	Web ranking, influence analysis
Degree	Number of direct connections	Activity level, popularity
Betweenness	Control over information flow	Identifying brokers, bottlenecks
Closeness	Average distance to all nodes	Speed of information spread
Eigenvector	Connections to important nodes	Influence in social networks

Community Detection Algorithms

Community detection identifies groups of nodes that are more densely connected to each other than to the rest of the graph:

Algorithm	Approach	Best For
Louvain	Modularity optimization	General purpose, scalable

Algorithm	Approach	Best For
Label Propagation	Neighbor consensus	Fast, near-linear time
WCC	Reachability	Finding disconnected subgraphs
SCC	Mutual reachability	Detecting cycles, rings
K-Core	Degree constraints	Finding cohesive cores

Pathfinding Algorithms

Pathfinding algorithms find routes or measure distances between nodes:

```
# Single shortest path
exec = conn.algo.shortest_path("node_a", "node_b")

# For all-pairs analysis, use NetworkX
exec = conn.networkx.run("all_pairs_shortest_path_length", "Customer", "distances")
```

---

11. Integration with DataFrames

Algorithm results stored in node properties can be easily exported to DataFrames for further analysis:

```
# Run algorithms
conn.algo.pagerank("Customer", "importance")
conn.algo.louvain("Customer", "community")
conn.networkx.betweenness_centrality("Customer", "betweenness")

# Export to Polars DataFrame
result = conn.query("""
    MATCH (c:Customer)
    RETURN
        c.customer_id AS id,
        c.name AS name,
        c.importance AS pagerank,
        c.community AS community,
        c.betweenness AS betweenness
""")

df = result.to_polars()

# Analyze in Polars
community_stats = df.group_by("community").agg([
    pl.col("pagerank").mean().alias("avg_pagerank"),
    pl.col("pagerank").max().alias("max_pagerank"),
    pl.col("betweenness").mean().alias("avg_betweenness"),
    pl.count().alias("members")
]).sort("avg_pagerank", descending=True)
```

```python
print(community_stats)

# Export to CSV for reporting
result.to_csv("/tmp/customer_analysis.csv")

# Export to Parquet for data pipelines
result.to_parquet("/tmp/customer_analysis.parquet")
```

---

12. Batch Algorithm Execution

For running multiple algorithms efficiently, batch them together:

```python
def run_full_analysis(conn, node_label: str, edge_type: str) -> dict:
    """Run comprehensive graph analysis and return summary."""
    import time

    results = {}
    start = time.time()

    # Define algorithms to run
    algorithms = [
        ("pagerank", "pr_score", {"damping": 0.85}),
        ("louvain", "community_id", {}),
        ("wcc", "component_id", {}),
        ("kcore", "k_degree", {}),
    ]

    for algo_name, prop_name, params in algorithms:
        algo_start = time.time()
        try:
            exec = conn.algo.run(
                algo_name,
                node_label=node_label,
                property_name=prop_name,
                edge_type=edge_type,
                params=params
            )
            results[algo_name] = {
                "status": "success",
                "nodes_updated": exec.nodes_updated,
                "duration_ms": exec.duration_ms
            }
        except Exception as e:
            results[algo_name] = {
                "status": "failed",
                "error": str(e)
            }
        print(f"  {algo_name}: {time.time() - algo_start:.1f}s")

    results["total_time_seconds"] = time.time() - start
    return results

# Usage
```

```python
analysis = run_full_analysis(conn, "Customer", "KNOWS")
print(f"Analysis completed in {analysis['total_time_seconds']:.1f}s")
```